

1. DAY 1 — What is RTL? HDL Basics & SystemVerilog Essentials:

Goal: Understand what RTL is, where it sits in the chip design flow, and learn the core **SystemVerilog** building blocks.

1.1. What is RTL?:

Register Transfer Level (RTL) is a way of describing digital hardware in terms of the flow of data **between registers** and the **operations** performed on that data during each clock cycle. The name literally describes what's being modeled: data is transferred from one register to another, possibly transformed by combinational logic along the way, on every clock edge.

Three abstraction levels, from most abstract to least abstract:

Level	What it describes	Example
Behavioral	What the circuit should do, without specifying hardware structure. Often not synthesizable.	“This block computes the average of two 8-bit numbers” written with high-level constructs (loops, delays used for algorithmic modeling)
RTL	How data moves between registers each clock cycle, and the combinational logic in between. Synthesizable.	<code>always_ff @(posedge clk) sum <= a + b;</code>
Gate-level netlist	The actual logic gates, flip-flops, and their exact interconnections — the output of synthesis.	AND2, OR2, DFF instances wired together

Table 1: Abstraction levels from behavioral to gate-level netlist.

RTL sits in the middle: abstract enough that a human can write and reason about it, concrete enough that a **synthesis tool** can mechanically translate it into gates.

Why RTL is the standard handoff format between design and verification teams:

RTL is precise enough to be simulated cycle-accurately (so a verification engineer can predict exact output values) yet still readable by humans (so intent can be reviewed, debugged, and documented). Gate-level netlists are too low-level to review efficiently; behavioral/algorithmic models are too abstract to guarantee synthesizability. RTL is the point of maximum leverage — it's the contract both teams work against.

The RTL-to-silicon flow:

```

1 RTL (SystemVerilog)
2   | synthesis (RTL -> gates, using a standard-cell library / PDK)
3   v
4 Gate-Level Netlist
5   | place & route (physical placement of cells, routing of wires)
6   v
7 Physical Layout (GDSII)
8   | fabrication

```

```

9      v
10     Silicon

```

Listing 1: The RTL-to-silicon flow.

- **Synthesis** reads RTL and a technology library (standard cells for a given process, e.g., ASAP7) and produces a gate-level netlist that implements the same logical behavior, optimized for timing/area/power.
- **Place & Route (P&R)** takes that netlist and decides exactly where each gate sits on the die and how wires connect them, producing the GDSII file used to fabricate the chip.

What a simulator does vs. what a synthesizer does — this distinction is critical:

	Simulator	Synthesizer
Input	RTL + testbench stimulus	RTL + technology library
Output	Waveforms / signal values over time	Gate-level netlist
Handles sim-only constructs (<code>initial</code> , <code>\$display</code> , <code>#10</code>)	Yes — these drive/observe simulation	No — these are simulation-only constructs, ignored or illegal in synthesis
Goal	Answer “does this behave correctly?”	Answer “how do I build this out of real gates?”

Table 2: Simulator versus synthesizer: inputs, outputs, and goals.

This is **why** not all valid **SystemVerilog** is synthesizable — a simulator will happily execute a `#10` delay or an `initial` block full of test stimulus, but a synthesis tool has no hardware equivalent for those constructs and will reject or ignore them.

1.2. HDL Fundamentals:

What a Hardware Description Language is, and why it is not a programming language: A programming language describes a sequence of instructions executed one at a time by a processor. An **HDL** describes a **structure** — a piece of hardware — where many pieces of logic operate **simultaneously and continuously**, all the time, forever (as long as the chip is powered). Writing `a = b + c;` in C means “compute this now, once.” Writing the equivalent in RTL means “there permanently exists a piece of hardware (an adder) whose output is always the sum of its inputs.” The mental model shift — from **sequence of steps** to **description of physical structure that behaves concurrently** — is the single biggest hurdle for newcomers to HDL.

Verilog vs. SystemVerilog:

- **Verilog** (IEEE 1364) was the original HDL, created in the 1980s, standardized in 1995. It’s sufficient for basic RTL and testbenches but is verbose and error-prone for verification work (e.g., weak typing, no native OOP, limited assertions).

- **SystemVerilog** (IEEE 1800) is a superset of Verilog that adds: stronger typing (`logic` type), more expressive procedural blocks (`always_comb`, `always_ff`, `always_latch`), interfaces, object-oriented programming (classes) for testbenches, native assertions (SVA), and functional coverage constructs (covergroups). For RTL design, SystemVerilog is mainly used for its clearer procedural blocks and the `logic` type; its OOP/class features are almost exclusively a verification-side (testbench) toolset.

Simulation vs. synthesis — not all valid SV is synthesizable. A construct is “synthesizable” if a synthesis tool can map it onto real hardware. Many SystemVerilog constructs exist purely to make **simulation** (testbenches) more powerful and have no synthesizable equivalent: delays (`#5`), `initial` blocks used for stimulus, `$display/$monitor`, dynamic arrays, classes, random number generation. As a rule of thumb: if you’re describing **stimulus, checking, or reporting**, it’s testbench-only; if you’re describing **hardware that must exist on-chip**, it must be synthesizable.

Concurrency — why hardware runs in parallel, not sequentially. A digital circuit is a physical object: every gate and flip-flop in it is doing something at every instant, independent of the “order” in which you wrote the code describing them. Two `always` blocks in the same module execute **concurrently** in simulation, mirroring the fact that the two pieces of hardware they represent operate at the same time in silicon. This is fundamentally different from software, where statement order dictates execution order.

Delta cycles and the simulation event queue (high-level understanding): A simulator doesn’t have real physical time resolution smaller than the discrete events it processes. Within a single simulation time step, signals can change multiple times as different concurrent processes react to each other (e.g., `always_comb` block A changes a signal that triggers `always_comb` block B). Each of these micro-steps, which happen at the **same** simulation time but in a defined order, is called a **delta cycle**. You don’t need to master delta-cycle scheduling in depth at this stage — the key takeaway is: **simulators resolve concurrent, interacting logic through an internal event queue, and this is why simulation order of same-time events can matter for subtle bugs (like race conditions between blocking and non-blocking assignments).**

1.3. SystemVerilog Module Structure:

module / endmodule — the basic reusable unit of hardware description. Everything about one hardware block (its interface and its internal behavior) lives between these two keywords.

```
1 module and_gate (  
2     input  logic a,  
3     input  logic b,  
4     output logic y  
5 );  
6     assign y = a & b;  
7 endmodule
```

Listing 2: Minimal two-input AND gate module.

Port declaration:

- `input` — a signal driven from **outside** this module, read-only inside it.
- `output` — a signal driven from **inside** this module, visible to whoever instantiates it.

- **inout** — bidirectional; used for things like tri-state buses (rare in typical RTL blocks, common in physical I/O pads).

Data types — **logic**, **wire**, **reg**:

Type	Meaning	When to use
wire	A pure connection (net) — must be continuously driven by something (an assign or a module output). Cannot be assigned inside a procedural block.	Structural connections between module instances
reg (legacy Verilog)	A variable that can be assigned inside a procedural block (always). Despite the name, does not necessarily imply a physical register/flip-flop — it can synthesize to combinational logic too. This ambiguity confuses beginners constantly.	Legacy Verilog code
logic (SystemVerilog)	A 4-state (0, 1, X, Z) type that can be used both as a net (driven by assign) and as a variable (driven inside always blocks) — except it cannot have more than one driver (so it can't directly replace a wire used for multi-driver buses).	The default choice for essentially everything in modern SV RTL

Table 3: Comparison of **wire**, **reg**, and **logic** data types.

In practice: modern **SystemVerilog** RTL uses **logic** almost everywhere and reserves **wire** only for nets that genuinely need multiple drivers (rare in typical student-level RTL).

parameter and **localparam** — making designs configurable:

- **parameter** — a compile-time constant that **can be overridden** when the module is instantiated (e.g., a FIFO's **DEPTH** or **WIDTH**).
- **localparam** — a compile-time constant that is **fixed** internally and cannot be overridden from outside (e.g., derived values like `$clog2(DEPTH)` for pointer width, or internal state encodings).

```

1 module sync_fifo #(
2     parameter WIDTH = 8,
3     parameter DEPTH = 8
4 ) (
5     ...
6 );

```

```

7     localparam PTR_WIDTH = $clog2(DEPTH);
8     ...
9 endmodule

```

Listing 3: Parameterized `sync_fifo` module skeleton.

Multi-bit signals and part-selects:

- `logic [7:0] data;` declares an 8-bit vector, bit 7 down to bit 0 (MSB-first convention, very common).
- `data[3:1]` selects bits 3 down to 1 (a 3-bit slice).
- `data[0]` selects a single bit.

Signed vs. unsigned: By default, `logic` vectors are **unsigned**. Declaring `logic signed [7:0] data;` makes **SystemVerilog** interpret the vector using two's-complement rules for arithmetic and relational operators (sign extension on assignment, sign-aware comparisons). Get this wrong and comparisons/arithmetic silently produce wrong results — always be deliberate about signed vs. unsigned, especially at module boundaries.

1.4. Concurrent vs. Procedural Code:

This is arguably the most important conceptual section of Day 1 — nearly every **FSM** bug traces back to misunderstanding these constructs.

assign — continuous assignment. Describes a piece of **pure combinational logic** that is always active — the moment any input changes, the output re-evaluates. Used for simple combinational logic outside procedural blocks (e.g., driving `full/empty` from a count register).

```

1 assign full  = (count == DEPTH);
2 assign empty = (count == 0);

```

Listing 4: Continuous assignment example for FIFO flags.

always_comb — combinational logic block. A **SystemVerilog**-specific procedural block (safer replacement for Verilog's generic `always @(*)`) that automatically re-evaluates whenever **any** signal read inside it changes. Models a network of logic gates with no memory — the output is a pure function of the current inputs.

```

1 always_comb begin
2     case (state)
3         IDLE: next_state = (din) ? S1 : IDLE;
4         ...
5     endcase
6 end

```

Listing 5: Combinational next-state logic sketch.

always_ff — sequential logic block. Models a **clocked** hardware element (flip-flops/registers). The block only executes on the specified clock edge (and reset edge, if asynchronous). This is the **only** place state should be updated in clean RTL style.

```

1 always_ff @(posedge clk or negedge rst_n) begin
2     if (!rst_n) state <= IDLE;
3     else      state <= next_state;
4 end

```

Listing 6: State register with asynchronous active-low reset.

always_latch — when it appears, and why it’s usually unintended. A latch is a level-sensitive (not edge-triggered) storage element. `always_latch` explicitly tells the tool “I mean to build a latch here.” In nearly all cases, a latch shows up **by accident** — the classic bug is an incomplete `case` statement inside `always_comb` (missing a `default`, or missing an `else`), where the tool must “remember” the previous value for the unhandled condition, which requires a latch. **Inferred latches are almost always bugs** in synchronous digital design and should be treated as errors to fix, not an available design style.

Sensitivity lists — @(posedge clk), @(*), and what they mean:

- `@(posedge clk)` — this block reacts **only** to the rising edge of `clk` (used for sequential logic).
- `@(*)` (legacy Verilog equivalent of `always_comb`’s implicit sensitivity) — reacts to **any** change of **any** signal read inside the block. `always_comb` computes this automatically and correctly, which is why it is preferred over manually written `@(*)` or explicit signal lists (which are easy to get wrong and forget a signal).

Summary table:

Construct	Models	Triggered by	Typical use
<code>assign</code>	Combinational logic	Any RHS signal change	Simple continuous logic, flag derivation
<code>always_comb</code>	Combinational logic	Any signal read inside, changes	Next-state logic, output logic
<code>always_ff</code>	Sequential logic (registers)	Clock edge (+ optional reset edge)	State register, counters, any stored value
<code>always_latch</code>	Level-sensitive latch	Level of control signal	Almost never intentional in this program

Table 4: Summary of SystemVerilog concurrent and procedural constructs.

1.5. Operators & Expressions:

Category	Operators	Notes
Bitwise	<code>&</code> , <code> </code> , <code>^</code> , <code>~</code>	Operate bit-by-bit across a full vector: <code>4'b1010 & 4'b1100 = 4'b1000</code>

Logical	<code>&&, , !</code>	Treat the entire operand as a single true/false value (non-zero = true); result is always 1-bit. Used in <code>if</code> conditions.
Reduction	<code>&a, a, ^a</code>	Collapse a whole vector into a single bit by applying the operator across all bits, e.g., <code>&a</code> is 1 only if every bit of <code>a</code> is 1 (useful for “all-ones” checks); <code>^a</code> computes parity.
Relational	<code>==, !=, ===, !==</code>	<code>==/!=</code> are 4-state-aware logical equality (result can be X if operands contain X/Z). <code>===/!==</code> are case equality — compare X and Z bits literally, used mainly in testbenches for exact bit-pattern checks, not typically used in synthesizable RTL.
Concatenation / Replication	<code>{a, b}, {4{a}}</code>	<code>{a, b}</code> joins bit vectors together (e.g., <code>{1'b0, byte}</code> zero-extends). <code>{4{a}}</code> repeats <code>a</code> four times — useful for sign/zero extension patterns.
Shift	<code><<, >>, <<<, >>></code>	<code><</>></code> are logical shifts (fill with 0). <code><<</>>></code> are arithmetic shifts — <code>>>></code> sign-extends for signed operands (preserves the sign bit), which matters a great deal for signed arithmetic correctness.

Table 5: SystemVerilog operator categories and semantics.

Common beginner trap: confusing `&` (bitwise AND across all bits, returns a vector) with `&&` (logical AND, returns a single bit) — e.g., `a & b` on 8-bit vectors returns an 8-bit result, while `a && b` returns 1 bit representing “is `a` non-zero AND is `b` non-zero.”

1.6. Day 1 — Key Takeaways:

- RTL is the sweet spot between human-readable design intent and machine-synthesizable precision — it’s **the** handoff artifact between design and verification.
- Hardware is inherently concurrent; HDL code describes structure, not a sequence of steps.
- `logic` is the default type for modern SV; `always_comb` for pure combinational logic, `always_ff` for anything clocked, and inferred latches (via incomplete `always_comb` cases) are bugs to eliminate.
- Not everything that simulates will synthesize — delays, `initial` stimulus blocks, and `$display` are simulation-only.

2. DAY 2 — FSM Theory: Moore, Mealy, State Encoding & Transition Tables:

Goal: Deeply understand **Finite State Machines** — the backbone of most RTL control logic.

2.1. What is a Finite State Machine?:

A **Finite State Machine (FSM)** is a mathematical model of a system that can be in exactly one of a **finite** number of **states** at any time, moves between states based on **inputs**, and produces **outputs** based on its state (and possibly its inputs).

The four components of any FSM:

1. **States** — the distinct “modes” or “memory conditions” the system can be in (e.g., IDLE, S1, S10).
2. **Inputs** — external signals that influence which state comes next.
3. **Outputs** — signals the FSM produces, which may depend on the state alone or on state + inputs.
4. **Transitions** — the rules that say “if I’m in state X and input condition Y is true, go to state Z.”

Why FSMs dominate control logic: almost any piece of hardware whose behavior depends on **history** (what happened before, not just what the inputs are right now) needs memory of “where it is” in a sequence of operations — that’s exactly what an FSM’s state provides. A combinational circuit alone can’t remember anything.

Real-world hardware examples: a **UART** receiver (waiting for start bit → sampling data bits → checking stop bit), a traffic light (cycling through timed phases), a **USB** controller (managing protocol handshake phases), a bus arbiter (deciding which requester gets the bus next, remembering whose turn is next for fairness).

2.2. Moore vs. Mealy:

	Moore	Mealy
Output depends on	Current state only	Current state AND current inputs
Output timing	Changes only on a clock edge (registered, one state per output value)	Can change immediately when an input changes, within the same clock cycle (combinational path from input to output)
Glitch risk	Low — output is a registered function of state	Higher — output is combinational, so it can glitch if inputs are noisy/change asynchronously
Gate count	Sometimes needs more states to capture the same behavior (an output condition might need to be split into a dedicated state)	Sometimes needs fewer states (input directly shapes output without a dedicated state)

Typical use	Most RTL control logic — favored for predictability and cleaner timing closure	Situations needing faster reaction to input (output must reflect an input within the same cycle)
-------------	--	--

Table 6: Moore versus Mealy finite state machines.

When to choose Moore (the common default for most RTL control logic): Moore machines produce more predictable, glitch-free timing because the output only changes synchronously with the clock — this makes timing analysis and downstream logic easier to reason about. Most instructional and real RTL control blocks default to Moore unless there's a specific reason (e.g., strict protocol timing) to need Mealy's faster response.

How to convert a Mealy FSM to a Moore FSM: In general, you can convert by “absorbing” the input-dependent part of the output into new states — i.e., splitting a state that has multiple different Mealy outputs into multiple Moore states (one per distinct output value), at the cost of extra states.

2.3. State Encoding:

State encoding is the scheme used to represent each abstract “state” as concrete bits stored in flip-flops.

Encoding	Description	Trade-offs
Binary	States numbered 0, 1, 2, ... using $\lceil \log_2(N) \rceil$ bits for N states	Fewest flip-flops (compact) but next-state decode logic can be more complex (more gates between flip-flops), potentially slower
One-hot	One flip-flop per state ; exactly one bit is HIGH at any time	Uses more flip-flops, but next-state/output decode logic is simpler and often faster (each state has a dedicated wire) — commonly preferred in FPGAs where flip-flops are cheap and speed matters
Gray	Adjacent states (in the sequence they're normally traversed) differ by exactly one bit	Minimizes simultaneous multi-bit switching — reduces switching noise and glitch risk, useful in low-power or noise-sensitive designs, and especially valuable for safely crossing clock domains

Table 7: State encoding schemes and their trade-offs.

How synthesis tools choose encoding: Most synthesis tools pick an encoding automatically based on optimization goals (area vs. speed) unless directed otherwise via `/* synthesis enum */`-style pragmas or explicit tool directives/constraints — designers can override the default when they have a specific reason (e.g., forcing one-hot for speed).

Trade-offs summary: speed (one-hot often wins), area (binary often wins — fewer flip-flops), power (Gray can win — less switching), debuggability (one-hot is easiest to read in a waveform viewer — you can see exactly which state bit is high; binary is more compact but requires decoding to read).

2.4. Transition Tables & State Diagrams:

Building a state transition table: a table with columns **Current State | Input | Next State | Output**. Every reachable combination of (current state, input) must map to exactly one (next state, output) row — this table is the **complete specification** of the FSM's behavior and should be built and verified **before** writing any RTL.

Drawing a state diagram:

- **Circles** (or boxes) = states.
- **Arrows** = transitions, each labeled with the input condition that causes it (and, for Mealy machines, the output produced).
- The diagram is a visual version of the same information in the transition table — building both and cross-checking them catches mistakes early.

Verifying a transition table against a state diagram: for every arrow drawn, there should be a matching row in the table, and vice versa — no arrows without a table row, no table row without a corresponding arrow. This cross-check is a cheap and effective way to catch missing transitions before coding.

Identifying unreachable states and invalid state handling: an **unreachable state** is one that no transition ever leads into — it's dead logic, usually a mistake. An **invalid state** is one that can occur in hardware (e.g., due to a glitch, uninitialized flip-flops on power-up before reset, or an encoding scheme with unused bit patterns) but was never intended by the designer — every FSM implementation should explicitly define what happens if the state register somehow lands there (typically: go to a safe/reset state).

Self-loops, reset states, and default state handling:

- A **self-loop** is a transition from a state back to itself (e.g., “stay in IDLE while input is 0”).
- The **reset state** is the state the FSM must always start in immediately after reset — this must be explicit and unambiguous in both diagram and RTL.
- **Default handling** — the **case** statement's **default:** branch should route any unexpected/invalid state safely back to a known state (this is also what prevents inferred latches, see [section 2.6](#)).

2.5. Two-Block FSM Style in SystemVerilog:

The recommended, clean way to code any FSM is with exactly **two** procedural blocks:

Block 1 — State register (sequential, `always_ff`): this is the **only** block that updates the actual state flip-flops.

```
1 always_ff @(posedge clk or negedge rst_n) begin
2     if (!rst_n) state <= IDLE;
3     else      state <= next_state;
4 end
```

Listing 7: Block 1: state register (sequential).

Block 2 — Next-state & output logic (combinational, `always_comb`): computes what the **next** state should be, and (for Moore) what the outputs are, purely as a function of the current state (and inputs, for next-state logic).

```

1 always_comb begin
2     next_state = state;          // safe default -- avoids latches
3     case (state)
4         IDLE: if (din) next_state = S1;
5         S1:   next_state = din ? S1 : S10;    // example only -- see Day 4 for
        ↪ the real detector logic
6         default: next_state = IDLE;
7     endcase
8 end

```

Listing 8: Block 2: next-state logic (combinational) with safe default assignment.

Why two blocks are cleaner than one monolithic block: separating “what state am I in” (pure memory, sequential) from “what should happen next” (pure logic, combinational) means each block has a single, clear responsibility. It also avoids a very common class of bugs where combinational decision logic accidentally gets tangled with clocked state updates (e.g., accidentally using blocking assignments in a sequential block, or accidentally creating extra unintended registers).

Proper state type declaration:

```

1 typedef enum logic [1:0] {
2     IDLE = 2'b00,
3     S1   = 2'b01,
4     S10  = 2'b10,
5     S101 = 2'b11
6 } state_t;
7
8 state_t state, next_state;

```

Listing 9: Enumerated state type declaration using `typedef enum`.

Using `typedef enum` (rather than raw parameters/localparams for each state) gives you named, type-checked states — the simulator/linter can catch typos and illegal assignments, and waveform viewers display the state **names** instead of raw bits, which is enormously helpful for debug.

Reset handling — synchronous vs. asynchronous:

- **Asynchronous reset** (`always_ff @(posedge clk or negedge rst_n)`): the reset takes effect **immediately**, independent of the clock — useful to guarantee a known state even if the clock isn’t toggling yet, but can create subtle timing issues (reset release timing / “reset recovery/removal” timing needs care).

- **Synchronous reset** (`always_ff @(posedge clk) if (!rst_n) ...`): reset only takes effect **on** a clock edge — simpler timing analysis, but the design must have a running clock for reset to take effect, and can need extra logic if reset must be guaranteed even without a clock.
- Which to use is a design/methodology choice; the important thing for this program is to be **consistent** and to know **which one you used and why**.

2.6. Common FSM Bugs:

Bug	Cause	Symptom
Inferred latch	Incomplete <code>case</code> in <code>always_comb</code> (missing <code>default</code> , or an <code>if</code> without an <code>else</code>)	Simulator/synthesis warning about latch inference; unpredictable hold-over behavior
Undefined behavior on illegal state	Missing <code>default</code> in the <code>case</code> statement	FSM can behave unpredictably if it ever reaches a state not explicitly listed
Output registered one cycle late	Confusing Moore timing (output reflects the state you're currently in, itself a registered value) with Mealy timing (output can react combinationally, same-cycle, to an input)	Detected/output pulse appears a cycle later (or earlier) than expected
State register not reset properly	Missing or incorrect reset logic in the sequential block	FSM starts in an unknown/random state after power-up, or reset condition not connected correctly
Overlapping detection errors	Not handling the case where the tail of one match can be the start of the next match (sequence detectors)	Detector misses valid overlapping occurrences of the target sequence

Table 8: Common FSM bugs, their causes, and symptoms.

Practical debugging habit: whenever an FSM output timing looks “off by one cycle,” the first thing to check is whether you meant Moore (state-only) or Mealy (state+input) behavior, and whether your RTL actually implements the one you intended.

2.7. Day 2 — Key Takeaways:

- **FSM** = states + inputs + outputs + transitions. Build the transition table and diagram **before** any RTL.
- **Moore** = output depends on state only (default choice, predictable timing). **Mealy** = output depends on state + input (faster reaction, more glitch-prone).
- **Encoding choice** (binary/one-hot/Gray) trades off area, speed, power, and debuggability.
- **Two-block style** (`always_ff` for state register, `always_comb` for next-state/output logic) is the clean, standard way to write FSMs in SystemVerilog.

- **Incomplete** case statements are the #1 source of inferred-latch bugs — always default every case.

3. DAY 3 — DV Fundamentals: What is Verification and Why Does It Exist?:

Goal: Understand what **design verification** is, why it is a separate discipline from design, and the core concepts and vocabulary.

3.1. Verification vs. Validation:

- **Verification** answers: “*Are we building the design right?*” — does the RTL correctly implement its own specification? This is an internal correctness check against a written contract.
- **Validation** answers: “*Are we building the right design?*” — does the specification itself actually satisfy what the customer/end-application needs? This is an external fitness-for-purpose check.

These happen at different stages and by different people because they answer fundamentally different questions — a design can be perfectly verified (matches its spec exactly) and still fail validation (the spec itself was wrong or incomplete for the real use case).

The cost-of-bug-discovery curve (the “10x rule”): a bug found at the RTL/simulation stage might cost a trivial amount of engineering time to fix. The *same* bug, if it escapes to gate-level or post-silicon stages, can cost roughly an order of magnitude more at each subsequent stage — because fixing it may require re-running synthesis, P&R, timing closure, or in the worst case, an actual chip re-spin (which costs real fabrication money and months of schedule). This is the single biggest reason verification exists as a rigorous, resourced discipline rather than an afterthought.

Famous chip bugs from insufficient verification (*useful cautionary examples — worth researching for your own KB entry*):

- **Pentium FDIV bug (1994):** a floating-point division error caused by a flawed lookup table, found *after* the chip shipped — led to a costly recall.
- **Ariane 5 (1996, software/embedded, frequently cited alongside hardware verification lessons):** a numeric overflow in reused code caused a catastrophic rocket failure shortly after launch — illustrates how insufficient testing of edge cases (not necessarily “hardware” per se, but the verification-mindset lesson is identical) can have severe consequences.

3.2. The Verification Problem:

Why you can’t simulate every possible input combination: for any design with N flip-flops and M input bits, the number of possible states and input combinations grows exponentially ($2^{(N+M)}$). Even modest designs quickly have more possible states/input sequences than could ever be exhaustively simulated in a human lifetime. This is called **state space explosion**.

Coverage-driven verification: because exhaustive simulation is impossible, the field adopted a pragmatic standard: define, in advance, the specific set of scenarios that *matter* (coverage goals),

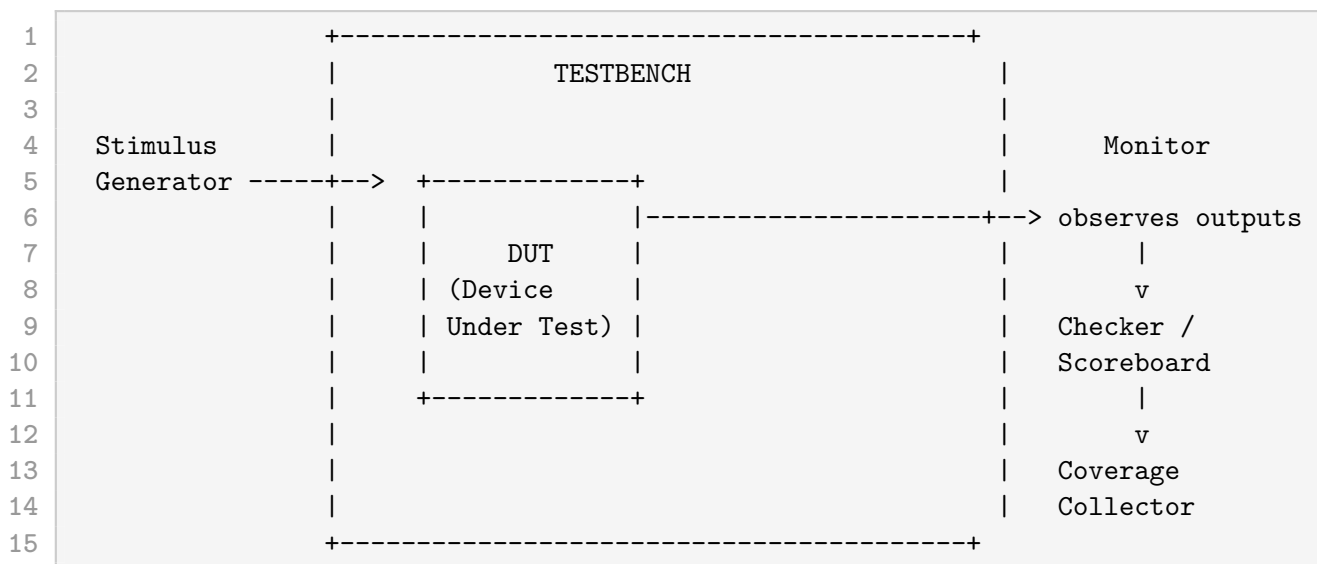
and verification is considered “done” when those defined goals are met — not when the team simply runs out of time or ideas. This turns an open-ended, unbounded task into a measurable, closable one.

Finding a bug vs. proving correctness: simulation-based verification can *find* bugs (a failing test proves the design is wrong) but can never *prove* the absence of all bugs (a passing test only proves that particular scenario worked). This is a fundamental asymmetry — passing tests build confidence, they don’t constitute mathematical proof.

Formal verification vs. simulation (high level): **formal verification** uses mathematical techniques (**model checking, theorem proving**) to *exhaustively* prove that certain properties hold for *all* possible inputs/states — a genuine proof, not sampled testing. It is extremely powerful for certain classes of properties (e.g., “these two signals can never both be high”) but doesn’t scale to prove full functional correctness of large, complex designs the way simulation-based coverage can be applied broadly. In practice, professional teams use both: formal for targeted critical properties, simulation for broad functional coverage.

3.3. Testbench Anatomy:

A **testbench** is the surrounding environment built specifically to exercise and check a design.



Listing 10: High-level testbench structure wrapping the DUT.

- **DUT (Device Under Test):** the RTL block being verified — the thing everything else exists to exercise and check.
- **Stimulus generator:** produces the input sequences that drive the DUT (can be directed — hand-written — or randomized within constraints).
- **Monitor:** passively *observes* DUT inputs/outputs without ever driving or interfering with them — it just watches and reports what happened.
- **Checker / Scoreboard:** compares what the DUT actually produced against what it *should* have produced (a “golden”/reference expectation), and flags mismatches.
- **Coverage collector:** tracks which scenarios have actually been exercised during simulation, so the team can measure progress toward coverage closure.

How these connect: the testbench *wraps* the DUT — the DUT itself is never modified for verification purposes; all the intelligence (stimulus, checking, coverage) lives in the surrounding testbench code, keeping the DUT as a clean, reusable, synthesizable block.

3.4. Types of Tests:

Type	Description	Best for
Directed	Hand-crafted, specific input sequences targeting one exact behavior	Predictable, reproducible checks; corner cases you can specifically imagine
Random / constrained-random	Inputs generated automatically, within legal constraints, often with a seed for reproducibility	Broad coverage; finding scenarios/interactions engineers didn't think to write by hand
Regression	Running the <i>entire</i> existing test suite together, typically after any design/RTL change	Catching newly introduced bugs that break previously-working behavior
Smoke test	A minimal directed test that just confirms the design isn't obviously broken (basic sanity)	Fast first check after any change, before investing in deeper testing

Table 9: Types of tests used in design verification.

3.5. The Verification Plan:

A **verification plan** (or “test plan”) is the document that lists *what* needs to be verified and *how* — written before any testbench code exists.

- **Feature:** a named, independently testable behavior of the DUT (e.g., “full flag asserts when count equals depth”).
- **Test case:** a specific scenario designed to exercise one or more features.
- **Priority:** classifies how critical a test is —
 - **P0 (must pass):** design is fundamentally broken without it; blocks sign-off.
 - **P1 (high importance):** important, generally required for sign-off, may have narrow exceptions/workarounds.
 - **P2 (nice to have):** adds confidence but isn't a blocker.
- **Status:** Not Started / In Progress / Pass / Fail — tracks where each test currently stands.

Why the plan is written before testbench coding: writing the plan first forces the team to think through *what correctness means* and *what could go wrong* independent of, and before being biased by, how the RTL happens to be implemented. It also lets the plan be reviewed and agreed upon (by designers, other verification engineers, leads) before any implementation effort is spent — catching gaps in thinking is far cheaper on paper than after code is written.

3.6. Coverage Basics:

Code coverage — automatically collected by the simulator, tells you which parts of the *RTL code* were exercised:

- **Statement coverage** — was each line executed at least once?
- **Branch coverage** — was every branch of every `if/case` taken at least once?
- **Toggle coverage** — did every signal toggle between 0 and 1 at least once?
- **FSM state + transition coverage** — was every state entered, and every transition arc traversed, at least once?

Functional coverage — written deliberately by the DV engineer, tells you which *meaningful scenarios* (from the spec’s point of view) were exercised:

- **Coverpoint**: a variable/expression whose values you want to track.
- **Bins**: the specific value(s) or ranges within a coverpoint that matter (e.g., “count == 0”, “count == DEPTH”, “count in between”).
- **Cross coverage**: tracks combinations of two (or more) coverpoints together (e.g., did we see all four combinations of `wr_en`/`rd_en`?).

Coverage closure: the deliberate process of driving stimulus (often adding new directed tests) specifically to hit any coverage bins that remain un-exercised, until all defined coverage targets are reached.

3.7. Assertions:

An **assertion** is a statement embedded in the design/testbench that must **always** be true during simulation — the tool checks it automatically, every relevant cycle, without any manual code needed to detect the violation.

- **Immediate assertions**: checked at the point they’re executed in procedural code (like a runtime `assert` in software) — evaluate a condition “right now.”
- **Concurrent assertions (SVA — SystemVerilog Assertions)**: describe temporal behavior across clock cycles (e.g., “if A happens, then B must happen within 3 cycles”) and are continuously, automatically checked by the simulator across time.

Examples (plain English):

- “Reset must fully deassert before any output is allowed to change.”
- “The `full` flag must never be asserted at the same time as `empty`.”

Why assertions are better than manual checking: a manually-coded check only fires where the engineer remembered to place it; an assertion, once written, fires **automatically on every relevant cycle for the entire simulation**, catching violations the engineer might never have thought to manually check for at that specific moment. This makes them extremely effective at catching subtle timing/protocol bugs with very little ongoing engineering effort.

3.8. Day 3 — Key Takeaways:

- Verification (“did we build it right?”) is a distinct discipline from validation (“did we build the right thing?”) — and finding bugs earlier (RTL stage) is dramatically cheaper than finding them later (post-silicon).
- Exhaustive testing is mathematically impossible for real designs — coverage-driven verification makes “done” a measurable, defined target instead of an open-ended guess.

- A testbench = DUT + stimulus generator + monitor + checker/scoreboard + coverage collector, all wrapped around an unmodified DUT.
- The verification plan (features, test cases, priorities) is written *before* any testbench code, and code coverage + functional coverage + assertions together give an objective picture of verification completeness.

4. DAY 4 — RTL #1: Sequence Detector (Overlapping 1011):

4.1. Quick Review — Sequence Detector Theory:

Overlapping detection: in an **overlapping sequence detector**, the tail end of one detected match is allowed to serve as the beginning of the *next* potential match — the detector does not need to “reset” and start fresh after a hit. The amount of overlap is found by comparing the pattern’s own suffixes against its own prefixes: for 1011, the longest proper suffix that is also a proper prefix is just 1 (length 1) — the suffix 11 is *not* equal to the prefix 10, so there is no length-2 overlap. This means only the **final bit** of a completed match carries over into the next potential match, not the whole tail. Concretely, in the stream 1011011, the match at bits 1–4 (1011) and the next match at bits 4–7 (1011) share exactly **one** bit (bit 4) — you must work this out bit by bit for any given pattern rather than assume a fixed overlap length, since it depends entirely on the pattern’s own structure.

Non-overlapping vs. overlapping — what changes in the state diagram: in a non-overlapping detector, after a match the FSM transitions back to IDLE unconditionally. In an overlapping detector, after a match the FSM must transition to whichever state correctly reflects *how much of the pattern is still “in progress”* based on the trailing bits of the match just completed — this usually means the “after match” transition target is *not* always IDLE.

Why the Moore output is registered, and the effect on detected-pulse timing: because a **Moore finite state machine (FSM)**’s output depends only on the *current state*, and the state itself only updates on a clock edge, the **detected** pulse appears exactly one full state (i.e., is aligned with) the cycle where the FSM sits in whichever state represents “match just completed” — not the same cycle as the input bit that completed the match combinatorially. Getting this timing relationship right (and correctly predicting it before simulating) is the main technical challenge of this day’s experimental work.

4.2. Two-Block SV Implementation:

Declaring the state type:

```
1 typedef enum logic [2:0] {
2     IDLE, S1, S10, S101, DETECT
3 } state_t;
4 state_t state, next_state;
```

Listing 11: State type declaration

A true **Moore** implementation needs **five** states, not four: IDLE, S1, S10, S101 track “how much of 1011 has been seen so far” — nothing, 1, 10, 101 — and a fifth state, DETECT, is entered the cycle *after* the full sequence completes. Making **detected** a pure function of “am I in DETECT”

— rather than combinational OR-ing in the current input bit while still in S101 — is exactly what keeps this a Moore machine instead of accidentally sliding into **Mealy** behavior. This is a common trap: it's tempting to save a state by asserting **detected** combinational the instant **din** completes the pattern in S101, but that makes the output depend on state *and* input — i.e., Mealy — even though the block diagram still “looks like” the FSM you set out to build.

State register block:

```
1 always_ff @(posedge clk or negedge rst_n) begin
2     if (!rst_n) state <= IDLE;
3     else      state <= next_state;
4 end
```

Listing 12: State register (sequential) block

Next-state logic block:

```
1 always_comb begin
2     next_state = state;    // safe default -- avoids latches
3     case (state)
4         IDLE:    next_state = din ? S1      : IDLE;
5         S1:      next_state = din ? S1      : S10;
6         S10:     next_state = din ? S101    : IDLE;
7         S101:    next_state = din ? DETECT  : S10;
8         DETECT:  next_state = din ? S1      : S10;    // trailing '1' from the
        ↪ match can start a new one
9         default: next_state = IDLE;
10    endcase
11 end
```

Listing 13: Next-state logic (combinational) block

Output logic (Moore — a pure function of state, not of din):

```
1 assign detected = (state == DETECT);
```

Listing 14: Output logic (Moore output assignment)

Notice **detected** never appears inside the **case** statement above, and never depends on **din** directly — only on which state the FSM currently sits in. That is what makes this a genuine Moore machine rather than a Mealy machine wearing a Moore label.

(This is illustrative — verifying the exact transition table by hand, per the Experimental Work below, is the actual assignment task; don't treat this snippet as a substitute for building your own table.)

Synchronous vs. asynchronous reset — which to use and why: for this design, either is acceptable, but the choice must be **deliberate and documented**. **Asynchronous reset** (**negedge rst_n** in the sensitivity list) guarantees the FSM lands in **IDLE** immediately, independent of clock activity — a common choice for simple control FSMs like this one, since it removes any ambiguity about “did the clock tick yet” during power-up/reset sequencing.

4.3. Smoke Testbench Basics:

Minimal initial block to drive a clock and stimulus:

```

1 initial clk = 0;
2 always #5 clk = ~clk;    // 10ns period clock
3
4 initial begin
5     rst_n = 0; din = 0;
6     @(posedge clk); rst_n = 1;
7     // drive stimulus bits here, one per clock edge
8 end

```

Listing 15: Minimal testbench clock and reset stimulus

\$display and \$monitor for printing signal values:

- `$display(...)` prints once, at the point it's executed in the code.
- `$monitor(...)` prints automatically, once per simulation time step, *whenever any of its listed signals change* — very convenient for a continuous per-cycle log without needing to call it repeatedly.

Checking output by eye: for a smoke test, the goal isn't automated pass/fail checking (that comes later, in the full verification phase) — it's simply confirming, by reading the printed log or waveform, that **detected** pulses on exactly the cycle you predicted by hand-tracing the input sequence.

4.4. Day 4 — Key Takeaways:

- Overlapping detection requires carefully tracing what happens to the *tail* of a completed match — don't assume a blind "return to IDLE."
- Moore output timing is tied to *state*, not directly to the input bit that triggered the transition — the pulse appears once the FSM has moved into the "match complete" state.
- A smoke test's purpose at this stage is human-verified sanity checking (predict → simulate → compare by eye), not exhaustive automated verification — that's what the later Verification Block builds toward.

5. DAY 5 — RTL #2: Traffic Light Controller:

Goal: Implement an **FSM** with timed states driven by a down-counter datapath.

5.1. FSM + Datapath Pattern:

Why a pure FSM isn't enough when states have durations: a plain FSM transitions purely based on the values of its inputs — it has no built-in notion of "wait N clock cycles." To hold a state for a specific duration, the FSM needs a companion piece of hardware — a **counter** — that measures elapsed time, and the FSM's transition condition becomes "counter has reached zero" rather than an external input.

The counter-driven FSM pattern:

```

1   FSM controls: which state we're in (and what duration to load)
2   Counter controls: how long we stay there
3   FSM transitions only when: counter reaches zero ("done" condition)

```

Listing 16: FSM/counter interface pattern

This is a very common and reusable **RTL** pattern: separate the *control* (what state, what should happen) from the *datapath* (the mechanism that measures time/values), and let a simple **done** signal be the interface between them.

Why parameterizing state durations matters: hardcoding a literal number like 20 directly inside FSM logic makes the design inflexible (can't retarget for different clock frequencies or requirements) and, critically for this program, makes simulation painfully slow (waiting real simulated cycles for a full-scale duration). Using **parameter** for each duration means the same RTL can be simulated with tiny values (e.g., 4 cycles) for fast, easy-to-trace testbenches, and later re-targeted to real-world values without touching the logic.

5.2. Counter Datapath in SV:

Down-counter behavior: loads a starting value when a state is entered, decrements by 1 each clock cycle thereafter, and signals **done** (or similar) once it reaches zero.

```

1  always_ff @(posedge clk or negedge rst_n) begin
2      if (!rst_n)      count <= '0;
3      else if (load)   count <= duration;
4      else if (count != 0) count <= count - 1;
5  end
6
7  assign done = (count == 0);

```

Listing 17: Down-counter datapath in SystemVerilog

Load vs. decrement priority: load should take priority over decrementing (typically checked first in the **if/else if** chain), since entering a new state and beginning its countdown is a distinct event from continuing to count down within a state.

parameter for duration values: each state's hold time is declared as a **parameter** (e.g., `parameter MAIN_GREEN_TIME = 4`), so a testbench can be built and debugged quickly using small values before (optionally) being re-parameterized for realistic durations.

5.3. Output Encoding:

Driving multiple output bits from a state: the traffic light has several independent output signals (e.g., `main_green`, `main_yellow`, `side_green`, `side_yellow`, and their corresponding red signals) — all of these are derived purely from the *current state*, making this a clean example of **Moore-style** multi-bit output logic.

One-hot output style: for each state, exactly one "group" of lights should be active (e.g., only the main-road green LED and side-road red LED are on simultaneously) — this constraint itself is a natural candidate for an **assertion** later (`main_green` and `side_green` must never both be HIGH).

What to output during YELLOW states: this requires a specific design decision that must be documented, not left implicit — for instance, during `MAIN_YELLOW`, is the side road red (safe, standard) or something else? These specific decisions are exactly the kind of thing that must be written down explicitly in the eventual RTL specification (Day 7/8) so a verification engineer doesn't have to guess.

5.4. Day 5 — Key Takeaways:

- FSM + datapath is a fundamental, reusable RTL pattern: the FSM decides *what*, a counter (or other datapath element) decides *how long/how much*, and a simple `done` flag connects them.
- Always parameterize durations/sizes rather than hardcoding literals — this keeps simulation fast and the design reusable.
- Multi-bit Moore outputs (like traffic lights) are still purely a function of state — the discipline from Day 1/2 (`always_comb` for outputs, `always_ff` only for the state/counter register) still applies directly.

6. DAY 6 — RTL #3: Synchronous FIFO:

6.1. What is a FIFO and Why It Exists:

FIFO = First In, First Out — data comes out in exactly the same order it went in, like a queue of people. It's fundamentally a *datapath*-oriented design (its correctness hinges on how data and pointers move through memory) rather than a *control*-oriented design (like the previous two FSM-centric blocks).

Where FIFOs appear in SoC design:

- **Clock domain crossings (CDC):** when data must move between two parts of a chip running on *different* clocks, an (asynchronous) FIFO is a standard, safe mechanism to hand data across safely.
- **Rate matching:** when a producer and consumer operate at different, possibly variable, rates (e.g., a fast producer feeding a slower consumer), a FIFO absorbs the rate mismatch by buffering.
- **General buffering** between any producer and consumer that shouldn't be tightly coupled in timing.

Synchronous FIFO: both the write side and the read side use the **same clock domain** — this is significantly simpler than an asynchronous FIFO (which must handle two independent, unrelated clocks and needs careful CDC-safe pointer synchronization, typically via **Gray-coded pointers** — recall Day 2's encoding discussion). This program's FIFO is the simpler, single-clock version.

6.2. FIFO Architecture:

- **Memory array:** `logic [WIDTH-1:0] mem [0:DEPTH-1];` — a small internal RAM-like structure, `DEPTH` entries, each `WIDTH` bits wide.
- **Write pointer (`wr_ptr`):** indicates the memory location where the *next* write will land.

- **Read pointer (`rd_ptr`):** indicates the memory location where the *next* read will come from.
- **Count register:** tracks how many valid (unread) entries currently sit in the FIFO — used to derive both **full** and **empty** without ambiguity (see [section 6.3](#)).
- **Pointer wraparound:** once a pointer reaches `DEPTH - 1` and needs to advance again, it must wrap back around to 0, since the memory array is finite and circular in how it's used.

6.3. Full & Empty Flag Logic:

- **empty:** `count == 0` — there is nothing valid to read.
- **full:** `count == DEPTH` — there is no room left to write.
- **Simultaneous read + write:** if both a valid read and a valid write happen in the *same* cycle, the count must stay **unchanged** (one entry leaves, one enters — net zero) — this must be handled as an *explicit* case in the logic, not assumed to “just work” from separate increment/decrement statements.
- **Write when full:** must be blocked entirely — attempting to write past capacity would silently overwrite unread data and corrupt FIFO ordering/content.
- **Read when empty:** must be blocked entirely — reading with nothing valid present would return garbage/stale data and should never be allowed to affect the pointers/count.

6.4. Pointer Arithmetic:

Why `wr_ptr == rd_ptr` alone cannot distinguish full from empty: both the “FIFO just reset (nothing written yet)” state and the “FIFO completely full, pointers have wrapped around and caught up to each other” state can produce `wr_ptr == rd_ptr` — the raw pointer equality is genuinely ambiguous between these two very different conditions.

Two standard solutions:

1. **Count register (the approach used in this program):** maintain an explicit counter of valid entries, independent of pointer comparison — simplest to reason about and implement correctly for a synchronous FIFO.
2. **Extra pointer bit:** widen each pointer by one extra bit beyond what's needed to address `DEPTH` locations; this extra bit effectively records how many times the pointer has “wrapped.” Comparing the extra bits (in addition to the address bits) then unambiguously distinguishes full from empty without needing a separate counter — more common in **asynchronous FIFO** designs where synchronizing a wide count register across clock domains would be problematic, but works for synchronous designs too.

Wraparound arithmetic:

```
1 wr_ptr <= (wr_ptr == DEPTH - 1) ? '0 : wr_ptr + 1'b1;
```

Listing 18: Circular-buffer pointer wraparound idiom

This ternary pattern — “if we just hit the last valid index, go back to zero; otherwise increment normally” — is the standard idiom for any circular-buffer pointer advance, and applies identically to `rd_ptr`.

6.5. Day 6 — Key Takeaways:

- A synchronous FIFO is a datapath-centric design: correctness lives in pointer/count book-keeping, not FSM state transitions.
- `full/empty` should be derived from an explicit count register (or an extra pointer bit) — never from raw pointer equality alone, which is inherently ambiguous.
- Simultaneous read+write, write-when-full, and read-when-empty are the three corner cases that separate a correct FIFO from a broken one — they must be handled explicitly, not left to chance.

7. DAY 7 — RTL Specification Writing:

7.1. What is an RTL Specification Document?:

The spec is **the contract** between the designer (who wrote the RTL) and the verifier (who will test it) — it defines what “correct behavior” actually means, independent of the RTL implementation itself.

A DV engineer must be able to write tests from the spec alone, without reading the RTL. This is the single most important test of a spec’s quality: if a verification engineer *needs* to read the RTL source code to understand what a signal is supposed to do, the spec has failed at its core job. The spec must be self-sufficient.

What happens when a spec is ambiguous: the designer and verifier end up with two different mental models of “correct” — the designer implements one interpretation, the verifier writes tests assuming another, and the bug/disagreement that surfaces during verification isn’t actually a design bug at all, it’s a *specification* failure. This is expensive to untangle because it isn’t caught by any single review — it only shows up when two independently-formed interpretations collide.

When the spec is written: after the design is functionally stable (so it accurately describes what was actually built, including all the corner-case decisions made along the way) but *before* verification planning begins (so the verification plan can be built directly and independently from the spec, not from assumptions about the RTL).

7.2. Sections of a Good RTL Spec:

Section	Purpose
Overview	What the block does, its role in the larger system/SoC, and the key design decisions made and why
Key Features	A bullet list of the main capabilities — “what can this block do?” at a glance
Interface Table	Every port: name, direction, width, clock domain, description
Microarchitecture	How the design works internally — datapath structure, control structure, parameters
FSM Specification	State encoding table, transition table, and an ASCII state diagram (for any FSM-based block)
Timing Diagrams	Key waveforms showing how signals relate to each other across clock cycles

Corner Cases	Every identified boundary condition and exactly how the RTL handles it
RTL Code	The actual SystemVerilog source, included at the end for full traceability

Table 10: Sections of a good RTL specification document

7.3. Writing a Good Interface Table:

Every port gets one row: **Port Name** | **Direction** | **Width** | **Clock Domain** | **Active Level** | **Description**.

- The **description** must state *when* a signal is asserted and *what it causes* — not merely restate the port’s name in different words. (“**rd_en**: asserted by the consumer for one cycle to request a read; when asserted while **empty** is low, **dout** updates with the oldest entry on the next cycle” is useful; “**rd_en**: read enable” is not.)
- **Active level matters and must be stated explicitly** — e.g., **rst_n** is active-low; failing to say so is exactly the kind of ambiguity that leads to a designer/verifier mismatch.
- **Width must exactly match the RTL** — any drift here (spec says one width, RTL implements another) is itself a spec-vs-RTL bug that Day 8’s cross-check is specifically designed to catch.

7.4. Writing a Good FSM Spec:

- **State encoding table:** State Name | Encoding | Description — documents exactly which bit pattern represents each state.
- **Transition table:** Current State | Condition | Next State | Output Actions — the same table built during design (Day 2/4), now formalized for the spec’s readers.
- **ASCII state diagram:** a text-based rendering of the state diagram (boxes for states, arrows for transitions, labels for conditions) so it can live directly inside a plain-text or lightly-formatted document.
- **Completeness check:** every state must be shown as reachable, and every transition arrow must have its condition explained — an FSM spec with unexplained transitions is incomplete.

7.5. Writing Good Corner Cases:

A **corner case** is a boundary condition where the design could plausibly fail if it isn’t handled explicitly and deliberately.

How to find corner cases: systematically think through what happens at the *edges* of every parameter and every signal — the minimum value, the maximum value, the moment of transition between two conditions, and any combination of two “edge” conditions happening simultaneously.

FIFO example corner cases (directly reusable from Day 6):

- Write when full
- Read when empty
- Simultaneous read+write while full
- Simultaneous read+write while empty

- Reset occurring mid-transaction

Corner case table format: Scenario | Expected Behavior | How it is Handled in RTL — each row should be specific enough that a verification engineer could write a directed test straight from it, without needing any additional clarification.

7.6. Day 7 — Key Takeaways:

- The RTL spec is a contract, not documentation-for-documentation's-sake — it must let a DV engineer write correct tests *without ever reading the RTL source*.
- Every interface table row needs a *behavioral* description (when/what-it-causes), not a re-stated name — and active-level/width must be exact.
- Systematically identifying corner cases (edges of every parameter/signal, and combinations of edges) is a skill in itself, and the resulting table becomes a direct input to the verification plan (Day 9 onward).

8. DAY 8 — Combined RTL Specification Document:

Goal: Produce one professional RTL_Specifications.docx covering all three designs.

8.1. Consistency Across Multiple Designs:

Using a consistent template across all three spec sections: each of the three RTL specs (Sequence Detector, Traffic Light, Sync FIFO) should follow the *exact same section structure* (Overview → Key Features → Interface Table → Microarchitecture → FSM Specification → Timing → Corner Cases → RTL Code) — this consistency is what makes a multi-design spec document feel professional and lets a reader navigate any section of any design using the same mental map, rather than re-learning the document's organization for each block.

Naming conventions must match the RTL exactly: every signal name, state name, and parameter name written in the spec must be character-for-character identical to what appears in the actual .sv files. Any drift here silently reintroduces the exact ambiguity problem the spec exists to prevent (see Day 7.1) — a verification engineer trusting the spec's naming would write a testbench referencing a signal that doesn't actually exist in the RTL, or means something subtly different.

Going back to fix RTL issues that spec writing reveals: the act of writing a rigorous, complete spec very often surfaces problems in the RTL itself that weren't obvious while coding — inconsistent naming, mismatched widths, missing or unclear reset values, corner cases that were never actually handled in the logic despite being "obviously" needed once written down explicitly. **This is expected and valuable**, not a failure — catching these issues during spec-writing (Days 7–8) is dramatically cheaper than catching them during verification (Days 9+) or, worse, not catching them at all. This is exactly why the program schedules a dedicated spec-writing block *before* verification planning begins.

8.2. Day 8 — Key Takeaways:

- A multi-design spec document's professionalism comes largely from *structural consistency* across all sections/designs, not from any one section in isolation.
- Naming must be exact and bidirectionally consistent between spec and RTL — treat any

mismatch as a bug to fix immediately.

- Expect (and welcome) the process of writing the spec to reveal real RTL issues — this is one of the most valuable side effects of rigorous documentation, not a distraction from it.

9. DAY 9 — Verification Planning Methodology, Test Categories & Req-IDs:

Goal: Understand how a professional verification plan is structured and how to organize tests into categories with traceability.

9.1. The Verification Plan as a Document:

The verification plan (test plan) is written **before any testbench code**, and answers three core questions:

1. **What** are we verifying? (the feature list)
2. **How** are we verifying it? (test categories, checker types, stimulus types)
3. **How do we know when we're done?** (entry/exit criteria, coverage targets)

Who writes it: typically the **DV lead**, with direct input from the RTL designer (who understands the intended behavior and known tricky corners best).

Who uses it: every DV engineer implementing tests against it, the project manager tracking progress, and the sign-off reviewer deciding whether the design is ready to proceed.

It's a living document: as bugs are found and design changes are made, the plan is updated — it is not a one-time artifact frozen at the start of verification.

9.2. Feature Identification:

A **feature** is a named, independently testable behavior of the **DUT**.

Where features come from: the RTL spec's interface table and corner cases sections (Day 7/8), protocol requirements (if any), and general design intent.

Feature granularity is a genuine skill — not too coarse, not too fine:

- Too coarse: `write_operation` — this bundles together many distinct behaviors (normal write, write when full, write during reset, etc.) into one unmeasurable blob.
- Too fine: `wr_ptr_increments_on_cycle_3` — this is an implementation detail, not an independently meaningful *behavior*, and is far too specific/brittle to be a useful feature.
- **Good examples:** `reset_initializes_all_outputs`, `full_flag_asserts_when_count_equals_data_preserved_on_simultaneous_read_write` — each names one clear, spec-derived, independently checkable behavior.

The feature list becomes the direct input to the rest of the verification plan.

9.3. Test Categories:

Category	Verifies
Reset / Initialization	The DUT starts in the correct state after reset
Functional	The core intended behavior — the “happy path”
Corner case / boundary	Behavior at the edges (full FIFO, zero-length sequence, single-cycle pulse)
Negative	The DUT correctly handles illegal inputs (write when full, read when empty, invalid state)
Stress / consecutive	Sustained operation over many cycles without error accumulation

Table 11: Verification test categories

9.4. Requirement IDs (Req-IDs):

A **Req-ID** is a unique identifier for each test case, e.g., RST_001, FUNC_003, CORNER_002.

- **Prefix** identifies the category: RST, FUNC, CORNER, NEG, STRESS — directly mirroring the categories in [section 9.3](#).
- **Sequential numbering** within each category.
- **Why Req-IDs matter:** they enable **traceability** — the ability to link a specific test to a specific feature, to a specific coverage item, and (later) to a specific bug report, forming an auditable chain from requirement to verification evidence.
- **Req-IDs never change once assigned** — even if a test is later removed (e.g., found to be redundant), its ID is **retired**, never reused for a different test. This preserves historical traceability (old bug reports, old coverage reports referencing that ID remain meaningful).

9.5. Entry and Exit Criteria:

- **Entry criteria:** conditions that must be true *before* verification work can begin — e.g., RTL is frozen (no further changes expected), the spec is complete, the simulation environment is set up and functional.
- **Exit criteria:** conditions that define verification as *done* — e.g., all P0 tests pass, 90%+ functional coverage achieved, zero open P0 bugs.
- **Why write these down in advance:** without explicit exit criteria, verification suffers from “just one more test” scope creep — there’s no objective stopping point, and the project can never cleanly declare sign-off. Writing criteria down *before* starting prevents this ambiguity and makes the eventual sign-off decision objective rather than a judgment call made under schedule pressure.

9.6. Traceability:

Traceability means every feature links to at least one test, and every test links to at least one coverage item.

- **A test with no coverage item cannot be measured** — there’s no way to confirm the scenario the test was meant to exercise was *actually* exercised, as opposed to the test passing for an unrelated/trivial reason.
- **A coverage item with no test** means that scenario was never deliberately targeted — it might get hit by accident during other testing, or it might never get hit at all, and nobody would necessarily notice.
- **Traceability matrix:** Feature → Req-ID → Coverage Link — this chain is the structural spine of a professional verification plan, and is exactly what Day 12’s traceability check audits at the end of the program.

9.7. Day 9 — Key Takeaways:

- The verification plan answers what/how/when-done, is written before any testbench code, and evolves as a living document.
- Feature granularity is a skill: not so broad it’s unmeasurable, not so narrow it’s an implementation detail rather than a behavior.
- Req-IDs (with category prefixes) are permanent, sequential identifiers that make traceability possible — and traceability (feature → test → coverage) is the backbone that makes a verification plan auditable and complete.

10. DAY 10 — Checker Types, Directed vs. Random Stimulus & Priority:

10.1. Checker Types:

Assertion (SVA):

- A property that must hold true on every relevant clock cycle throughout simulation.
- Best for: protocol invariants, timing rules, and signal relationships that should *always* be true (not just at specific test moments).
- Examples: “full and empty must never both be HIGH simultaneously”; “detected must be LOW whenever din has not completed the target sequence.”
- Fires automatically — no manual in-testbench checking code required.
- Concept-level syntax: `assert property (@(posedge clk) !(full && empty));`

Scoreboard:

- A testbench component that tracks what the DUT *should* produce (using a reference/“golden” model) and compares it against what the DUT *actually* produces.
- Requires a **reference model** — typically a simple software (non-RTL) representation of correct behavior, easy to write and trust precisely because it’s simple and separate from the DUT implementation.
- Best for: data integrity checks, output value correctness, and order-preserving designs (FIFOs, pipelines) — anything where “is this specific value correct” matters over many transactions.
- Example: a FIFO scoreboard stores every value written into a software queue; on every read, it checks that `dout` matches the oldest still-unread entry in that queue.

Checker with Expected Value:

- Simpler than a full scoreboard — used when the exact expected output is already known ahead of time because the stimulus is fully directed/controlled.
- Best for: directed corner-case tests, where “what should happen” can be predicted by hand before running the simulation.
- Example: after driving 1011 on `din`, check that `detected` goes HIGH on the specific next cycle predicted by hand-tracing (echoing Day 4’s exercise).

10.2. Directed vs. Random Stimulus:

Directed stimulus: the engineer writes the exact input sequence by hand.

- Pros: predictable, easy to debug (you know exactly what was driven), well-suited to P0 features and corner cases.
- Cons: limited coverage breadth — only exercises scenarios the engineer specifically thought to write.

Random / constrained-random stimulus: inputs are generated automatically within defined legal constraints.

- Pros: finds unexpected interactions/bugs the engineer didn’t think to test for; scales naturally to cover large input spaces without hand-writing every case.
- Cons: harder to debug (which specific input caused a failure requires more investigation); needs a scoreboard (or similarly automated checker) since there’s no pre-written expected value to compare against.
- Example constraint: randomize `wr_en` and `rd_en` independently, but *constrain out* illegal combinations (e.g., don’t randomly assert `wr_en` while `full` is already known to be true, if that’s defined as illegal input for the DUT).

Choosing between them — a practical rule of thumb:

- Reset behavior, corner cases, negative tests → **directed**.
- Broad functional behavior, general data integrity → **constrained-random**.
- Most real verification plans use **both**: directed tests to guarantee P0 confidence and hit specific known corners, random tests to close out broad functional coverage and surface unanticipated bugs.

10.3. Priority Levels:

Priority	Meaning	Example
P0 — Must Pass	Design is fundamentally broken without this; blocks sign-off entirely	Reset works correctly; basic write/read works; full/empty flags are correct
P1 — High Importance	Important for sign-off, may occasionally have narrow accepted workarounds	Simultaneous read/write handling; back-to-back operations without gaps

P2 — Nice to Have	Adds confidence but is not itself a sign-off blocker	Long stress runs; unusual-but-legal input patterns
--------------------------	--	--

Table 12: Verification priority levels

10.4. Matching Features to Methods:

Feature type	Recommended checker	Recommended stimulus
Protocol invariant (always true)	Assertion	(applies continuously, any stimulus)
Data correctness (output matches input, over time)	Scoreboard	Often paired with random/constrained-random for breadth
Specific timed sequence	Checker with expected value	Directed
Broad functional coverage	Scoreboard	Constrained-random
Negative / illegal input	Assertion (confirming it does <i>not</i> fire in normal operation)	Directed

Table 13: Matching feature types to checker and stimulus methods

10.5. Day 10 — Key Takeaways:

- Three checker types exist for three different jobs: assertions for always-true invariants, scoreboards for ongoing data-correctness against a reference model, and simple expected-value checks for fully-predictable directed scenarios.
- Directed stimulus gives predictability and control (great for P0/corners); random stimulus gives breadth and surprise-bug discovery (great for coverage closure) — professional plans use both together, not one instead of the other.
- P0/P1/P2 priority is about *sign-off risk*, not about how interesting or difficult a test is to write.

11. DAY 11 — Coverage Theory: Code Coverage, Functional Coverage & Covergroups:

Goal: Understand how coverage is measured, modeled, and used as the metric for verification completeness.

11.1. Why Coverage Exists:

The fundamental problem: how do you objectively know when you’ve tested *enough*? Without a defined metric, the honest answer becomes “when we ran out of time” — which is not remotely acceptable for something as expensive and hard-to-fix-after-the-fact as silicon.

Coverage gives an objective, measurable answer: verification is considered “done” (for a given target) when defined coverage goals are met — turning an otherwise subjective judgment call into a trackable, reportable number.

Coverage as a proxy for confidence, not a guarantee of correctness: high **coverage** does **not** prove the design is bug-free (a scenario can be “exercised” by a test that doesn’t actually check the result correctly). However, **low coverage does guarantee insufficient testing** — if a scenario was never exercised at all, it certainly wasn’t verified. This asymmetry (coverage is a necessary-but-not-sufficient signal) is important to internalize.

11.2. Code Coverage:

Automatically collected by the simulator — tells you which parts of the **Register Transfer Level (RTL)** code were actually exercised during simulation, with no manual instrumentation needed.

Metric	Question it answers	Why it matters
Statement coverage	Was this line of RTL ever executed?	A line never executed cannot possibly have been tested
Branch coverage	For every if/case , was every branch taken?	A branch never taken is an entirely untested logical path
Toggle coverage	Did each signal toggle between 0 and 1 at least once?	A signal stuck at 0 (or 1) the whole simulation could be silently wrong and nobody would notice without this check
FSM state coverage	Was every state entered at least once?	An unentered state is entirely unverified
FSM transition coverage	Was every arc in the state diagram traversed at least once?	The strongest FSM metric — a never-taken transition could hide a bug specifically in that condition’s logic
Condition coverage	For compound conditions (a && b), was every sub-condition independently exercised as both true and false?	Catches cases where a compound condition “passes” overall but one of its terms was never actually meaningfully varied

Table 14: Code coverage metrics tracked automatically by the simulator.

Target: 100% code coverage is treated as the *baseline* — it certifies “no dead code, no completely untested paths,” not “the design is correct.”

11.3. Functional Coverage:

Written deliberately by the **DV (Design Verification) engineer** — tells you which *meaningful scenarios* (from the perspective of the spec, not the code structure) were exercised. The simulator cannot infer this on its own; it has to be modeled explicitly.

- **Coverpoint:** a variable or expression whose values are tracked over the simulation, e.g., `coverpoint count` — tracks which values the FIFO’s count register actually took on.
- **Bins:** the specific values or ranges within a coverpoint that matter to the verification goal:

```

1 bins empty = {0};
2 bins full  = {DEPTH};
3 bins mid   = {[1:DEPTH-1]};
4

```

Listing 19: Example bins for a FIFO count coverpoint

- **Cross coverage:** tracks combinations of two or more coverpoints together, e.g., `cross wr_en, rd_en` — did the simulation actually exercise all four combinations (both off, write-only, read-only, both simultaneously)?
- **Assertion coverage:** tracks whether a specific assertion’s associated `cover property` ever actually fired during simulation — confirming the scenario the assertion is meant to guard against was genuinely exercised, not just that the assertion never happened to trip.

11.4. Coverage Closure:

Coverage closure is the deliberate process of reaching all defined functional coverage targets.

- **Typical target:** 90–100% functional coverage before sign-off, though the exact bar depends on company/project policy.
- **When a bin isn’t hit:** the response is to write a *directed* test specifically aimed at that exact scenario — this is a direct, practical link back to Day 10’s directed-vs-random discussion: random stimulus is good for broad coverage, but closing the last few stubborn holes is almost always a job for targeted, directed stimulus.
- **Coverage holes:** scenarios that random stimulus never happened to generate on its own — these need deliberate directed tests to fill, since relying on random chance to eventually hit them is unreliable and wastes simulation time.
- **Coverage report:** the simulator produces a report showing the percentage hit for each covergroup/bin, which becomes the concrete evidence used in exit-criteria sign-off (Day 9.5).

11.5. Linking Coverage to Tests:

- Every covergroup item should link to at least one test’s Req-ID.
- A coverage item with **no** linked test means it was never deliberately targeted — it might be hit purely by accident during unrelated testing, or it might never be hit at all, and there’d be no clear ownership or intent behind either outcome.
- The **Coverage Link** column in the test plan (e.g., “FUNC_003” written next to a coverage item) records which specific test is *responsible* for exercising that coverage item.
- This link is precisely what completes the traceability chain from Day 9.6 (Feature → Req-ID → Coverage Link) and makes the verification plan a fully auditable, complete document rather than a loose collection of separately-tracked lists.

11.6. Designing a Coverage Model:

Practical method for building a coverage model:

- Start from your **feature list** — every feature should map to at least one coverage item, so nothing in the plan is left unmeasured.

- **For FSM designs:** add bins for every state and every transition (directly mirroring FSM code coverage from [section 11.2](#), but modeled as *functional* intent rather than relying solely on the simulator’s automatic tracking).
- **For datapath designs:** add bins for boundary values (0, 1, max–1, max) and general mid-range values — directly echoing the corner-case thinking from Day 7.5.
- **For data signals:** add bins for all-zeros, all-ones, and alternating-bit patterns — these patterns are classic ways to catch stuck-bit or bit-order bugs.
- **For control signals:** add bins covering all combinations of related signals, using **cross coverage** (e.g., all four `wr_en`/`rd_en` combinations for the FIFO).

11.7. Day 11 — Key Takeaways:

- Code coverage (automatic, structural — did the *code* run) and functional coverage (deliberate, scenario-based — did the *meaningful behavior* happen) are complementary, not interchangeable — you need both.
- Coverpoints track a variable; bins define the specific values that matter; cross coverage tracks combinations of coverpoints together.
- Coverage closure is an active process: when a bin is missed, write a directed test to specifically target it — don’t just hope random stimulus eventually gets there.
- Every coverage item should trace back to a Req-ID, completing the Feature → Test → Coverage traceability chain from Day 9.

12. DAY 12 — Knowledge Base Completion & Self-Review:

Goal: Complete all documentation, review everything against the rubric, and ensure the repo is clean.

12.1. 12.1 Traceability Check:

This is the day the entire Feature → **Req-ID** → Coverage chain (built incrementally across Days 9–11) gets **audited end-to-end**:

- **Every feature** from your spec → must map to *at least one* Req-ID in your feature table. (If a spec-derived feature has no test at all, that’s a gap — go back and add one.)
- **Every Req-ID** → must map to *at least one* coverage item. (A test with no coverage link, per Day 11.5, can’t be objectively confirmed to have exercised what it was meant to.)
- **Every coverage item** → must link back to a Req-ID. (A coverage item with no owning test, per Day 11.5, was never deliberately targeted.)
- **Run through each of your three RTLs** and manually check that this chain is genuinely unbroken, in both directions, for every entry — not just spot-checked.

This traceability check is where the abstract “Feature → Req-ID → Coverage Link” concept from Day 9.6 stops being a diagram and becomes a concrete audit you perform on your own actual documents.

12.2. 12.2 Verification Metrics & Closure Thinking:

- **Review your priority assignments:** are *all* P0 features actually covered by at least one test and one coverage item? (P0 gaps are the most serious kind — recall from Day 10.3 that P0 failures/gaps block sign-off entirely.)
- **Check corner cases against Req-IDs:** are all the corner cases you identified back in your RTL specs (Day 7.5) actually captured as distinct Req-IDs in your verification plan? A corner case documented in the spec but never turned into a test is a silent gap.
- **Look for orphaned coverage items:** are there any coverage items with no test assigned to them? (Directly checking against the Day 11.5 principle.)
- **Preview thinking — what would come next:** what would a **DV engineer** actually need to do *next* to run these tests for real (write the actual **SystemVerilog** testbench code, build the reference model/scoreboard, set up the simulation environment)? This is explicitly framed as a preview of a *future* assignment — Days 1–12 (and 13–14) are about building the conceptual foundation, the RTL, and the *plan*; actually coding and running the testbenches is the next stage of the DV skillset beyond this program’s scope.

12.3. Day 12 — Key Takeaways:

- Traceability is not just a Day 9 concept to learn — it’s a concrete, bidirectional audit you must actually perform on your own three RTLs’ documentation before this program’s deliverables can be considered complete.
- P0 gaps and orphaned coverage items are the two specific things this self-review is designed to surface and eliminate.
- Completing this audit honestly (documenting gaps *and* fixing them) is itself part of what’s being evaluated — recall the program’s core rule from the outset: “produce real outputs even if incomplete, do not hide mistakes.”

13. How These Days Connect (Big Picture):

1	Days 1–3	Study Block 1	->	RTL fundamentals + FSM theory + DV vocabulary
2				(the concepts everything else depends on)
3	Days 4–6	RTL Design	->	Build the 3 RTLs using those concepts
4	Day 7	Study	->	Learn how to document what you built
5	Day 8	Documentation	->	Write the actual spec (contract for
		↪ verification)		
6	Days 9–11	Study Block 2	->	Learn how to PLAN verification against that spec
7	Day 12	Self-Review	->	Audit that the whole chain (spec -> plan ->
		↪ coverage)		
8				is complete and unbroken
9	Days 13–14	Wrap-up	->	Reflect and present

Listing 20: High-level flow connecting Days 1–14 of the program

The throughline across all 12 days is a single idea repeated at increasing levels of rigor: **every design decision must be explainable, and every claim of “this works” must be traceable back to something concrete** — a transition table, a spec section, a Req-ID, a coverage bin. That discipline *is* the difference between a hobbyist digital design exercise and professional RTL/DV engineering practice.

14. Quick Reference — Key Terms Across Days 1–12:

Alphabetized glossary of every term introduced in the notes above. Use this as a fast lookup while reviewing, or as a self-test — cover the extbfDefinition column and see if you can state each one from memory.

Term	Definition	Day(s)
<code>always_comb</code>	SV procedural block modeling combinational logic; auto-re-evaluates whenever any signal read inside it changes	1, 2
<code>always_ff</code>	SV procedural block modeling clocked/sequential logic (registers); executes only on the specified clock/reset edge	1, 2
<code>always_latch</code>	SV procedural block modeling a level-sensitive latch; usually appears unintentionally from an incomplete <code>always_comb</code> case	1, 2
Assertion	A statement embedded in design/testbench code that must always be true; checked automatically by the tool	3, 10
Asynchronous reset	Reset that takes effect immediately, independent of the clock edge	2, 4
Bins	The specific values or ranges within a coverpoint that matter for functional coverage	3, 11
Branch coverage	Code coverage metric: was every branch of every <code>if/case</code> taken at least once?	3, 11
Checker with expected value	Simple checker for directed tests where the exact correct output is known in advance	10
Code coverage	Simulator-collected metric measuring which parts of the RTL code were executed (statement, branch, toggle, FSM, condition)	3, 11
Concurrency	The principle that hardware blocks operate simultaneously, not in sequential program order	1
Condition coverage	Code coverage metric: for compound conditions, was every sub-condition independently true and false?	11
Corner case	A boundary condition where a design could fail if not explicitly handled (e.g., write when full)	6, 7
Count register	A register tracking the number of valid entries in a FIFO; used to derive <code>full/empty</code> unambiguously	6
Coverage closure	The process of driving stimulus (often via added directed tests) until all coverage targets are reached	3, 11
Coverage collector	Testbench component tracking which scenarios have been exercised during simulation	3
Coverage hole	A coverage bin that stimulus (often random) never happened to hit	11
Coverage-driven verification	Methodology where verification is “done” when defined coverage targets are met, not when time runs out	3

Coverpoint	A variable or expression whose values are tracked for functional coverage	3, 11
Cross coverage	Functional coverage tracking combinations of two or more coverpoints together	3, 11
Delta cycle	A micro-step within a single simulation time step, used to resolve interacting concurrent processes	1
Directed test	A hand-crafted test targeting one specific, predictable behavior	3, 10
Down-counter	A counter that loads a starting value and decrements to zero, signaling “done”; pairs with an FSM to implement timed states	5
DUT (Device Under Test)	The RTL block being verified by a testbench	3
Empty flag	FIFO signal asserted when <code>count == 0</code> ; nothing valid to read	6
Entry criteria	Conditions that must be true before verification work can begin (RTL frozen, spec complete, environment set up)	9
Exit criteria	Conditions that define verification as complete (all P0 pass, coverage target hit, zero open P0 bugs)	9
Feature	A named, independently testable behavior of the DUT	3, 9
Formal verification	Mathematical technique that exhaustively proves certain properties hold for all possible inputs/states	3
FSM (Finite State Machine)	A model with a finite set of states, inputs, outputs, and transitions between states	2
FSM + Datapath pattern	Design pattern where an FSM decides <i>what</i> happens while a counter/datapath element decides <i>how long/how much</i>	5
FSM coverage	Code coverage of state entry and transition traversal (state coverage + transition coverage)	3, 11
Full flag	FIFO signal asserted when <code>count == DEPTH</code> ; no room left to write	6
Functional coverage	DV-engineer-authored coverage tracking whether meaningful spec-level scenarios were exercised	3, 11
Gray encoding	State encoding where adjacent states differ by exactly one bit; minimizes switching noise	2
HDL (Hardware Description Language)	A language describing physical hardware structure/behavior, not sequential program instructions	1
Immediate assertion	An assertion checked at the point it executes in procedural code, like a runtime check	3
Interface table	Spec section listing every port: name, direction, width, clock domain, active level, description	7, 8

localparam	A compile-time constant that cannot be overridden from outside the module	1
logic	The default SystemVerilog 4-state data type usable as both net and variable	1
Mealy FSM	An FSM whose output depends on current state AND current input	2
Microarchitecture	Spec section describing internal datapath/control structure and parameters	7
Monitor	Testbench component that passively observes DUT signals without driving them	3
Moore FSM	An FSM whose output depends only on the current state	2
One-hot encoding	State encoding using one flip-flop per state; simpler decode logic, more flip-flops	2
Overlapping detection	Sequence-detector behavior where the tail of one match can begin the next match	4
parameter	A compile-time constant that can be overridden at module instantiation	1
Pointer wraparound	Circular-buffer behavior where a pointer resets to 0 after reaching $DEPTH - 1$	6
Priority (P0/P1/P2)	Classification of test criticality: must-pass, high-importance, nice-to-have	3, 10
Random / constrained-random stimulus	Automatically generated inputs within legal constraints, used for broad coverage	3, 10
Regression	Running the full existing test suite after a design change, to catch newly introduced bugs	3
Req-ID	A unique, permanent, category-prefixed identifier for a test case (e.g., FUNC_003)	9
Reset state	The state an FSM must always occupy immediately after reset	2
RTL (Register Transfer Level)	Hardware description in terms of data transfer between registers each clock cycle	1
Scoreboard	Testbench checker comparing DUT output against a reference/golden model over time	3, 10
Self-loop	A state transition that returns to the same state	2
Smoke test	A minimal directed test confirming a design isn't obviously broken	3, 4
State encoding	The bit-pattern scheme used to represent FSM states (binary, one-hot, Gray)	2
State space explosion	Exponential growth in possible states/inputs that makes exhaustive simulation impossible	3

Statement coverage	Code coverage metric: was each line of RTL executed at least once?	3, 11
Stimulus generator	Testbench component producing inputs to drive the DUT	3
SVA (SystemVerilog Assertions)	Concurrent assertions describing temporal behavior across clock cycles	3, 10
Synchronous FIFO	A FIFO where read and write sides share the same clock domain	6
Synchronous reset	Reset that only takes effect on a clock edge	2, 4
Synthesizable	Describes RTL constructs that a synthesis tool can map onto real hardware	1
Test case	A specific scenario designed to exercise one or more features	3, 9
Test categories	Reset/Initialization, Functional, Corner case, Negative, Stress — the standard test classification scheme	9
Testbench	The surrounding environment (stimulus + monitor + checker/scoreboard + coverage collector) built to exercise and check a DUT	3
Toggle coverage	Code coverage metric: did each signal toggle between 0 and 1 at least once?	3, 11
Traceability	The property that every feature links to a test, and every test links to a coverage item	9, 12
Traceability matrix	Table structure: Feature → Req-ID → Coverage Link	9, 12
Transition table	Table of Current State + Input → Next State + Output, fully specifying FSM behavior	2, 7
<code>typedef enum</code>	SystemVerilog construct for declaring a named, type-checked state type	2
Validation	Checking whether the specification itself meets the real-world need (“right design?”)	3
Verification	Checking whether the RTL correctly implements its own specification (“design built right?”)	3
Verification plan	Document defining what is verified, how, and when it’s considered done; written before testbench code	3, 9
<code>wire</code>	A net type requiring continuous driving; cannot be assigned inside a procedural block	1
Write/Read pointer	FIFO registers indicating the next memory location to write to / read from	6

Table 15: Quick reference glossary of key terms across Days 1–12